

Air University



Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028
Department of Cyber Security

USAMA ARSHAD | 247141
BS – Cyber Security – Fall – 24 (A)
Date of Submission: March 05, 2026

Lab Task # 05

1. Code:

```
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

// Encrypts plaintext using the Vigenère cipher
string vigenereEncrypt(const string &plaintext, const string &key)
{
    string ciphertext = "";
    int keyIndex = 0;
    int keyLen = key.length();

    for (char c : plaintext)
    {
        if (isalpha(c))
        {
            char base = isupper(c) ? 'A' : 'a';
            int shift = tolower(key[keyIndex % keyLen]) - 'a';
            ciphertext += (char)((c - base + shift) % 26 + base);
            keyIndex++;
        }
        else
        {
            ciphertext += c;
        }
    }

    return ciphertext;
}

// Decrypts ciphertext using the Vigenère cipher
string vigenereDecrypt(const string &ciphertext, const string &key)
{
    string plaintext = "";
    int keyIndex = 0;
    int keyLen = key.length();

    for (char c : ciphertext)
    {
        if (isalpha(c))
        {
            char base = isupper(c) ? 'A' : 'a';
            int shift = tolower(key[keyIndex % keyLen]) - 'a';
            int decrypted = (c - base - shift + 26) % 26;
            plaintext += (char)(decrypted + base);
            keyIndex++;
        }
    }
}
```

```
        else
        {
            plaintext += c;
        }
    }

    return plaintext;
}

int main()
{
    int choice;

    do
    {
        cout << "\n===== Vigenere Cipher Program =====" << endl;
        cout << "1. Encrypt" << endl;
        cout << "2. Decrypt" << endl;
        cout << "3. Exit" << endl;
        cout << "Enter your choice (1-3): ";
        cin >> choice;

        if (choice == 1)
        {
            // Encryption mode
            string plaintext, key;
            cout << "\n--- Encryption ---" << endl;
            cout << "Enter plaintext: ";
            cin.ignore(); // Clear newline from buffer
            getline(cin, plaintext);
            cout << "Enter key: ";
            getline(cin, key);

            if (key.empty())
            {
                cout << "Error: Key cannot be empty!" << endl;
                continue;
            }

            string ciphertext = vigenereEncrypt(plaintext, key);
            cout << "\nResult:" << endl;
            cout << "Plaintext: " << plaintext << endl;
            cout << "Key: " << key << endl;
            cout << "Ciphertext: " << ciphertext << endl;
        }
        else if (choice == 2)
        {
            // Decryption mode
            string ciphertext, key;
            cout << "\n--- Decryption ---" << endl;
            cout << "Enter ciphertext: ";
            cin.ignore(); // Clear newline from buffer
```

```
getline(cin, ciphertext);
cout << "Enter key: ";
getline(cin, key);

if (key.empty())
{
    cout << "Error: Key cannot be empty!" << endl;
    continue;
}

string plaintext = vigenereDecrypt(ciphertext, key);
cout << "\nResult:" << endl;
cout << "Ciphertext: " << ciphertext << endl;
cout << "Key: " << key << endl;
cout << "Plaintext: " << plaintext << endl;
}
else if (choice == 3)
{
    cout << "\nThank you for using Vigenere Cipher!" << endl;
    break;
}
else
{
    cout << "Invalid choice! Please enter 1, 2, or 3." << endl;
}

} while (choice != 3);

return 0;
}
```

Code Explanation:

- Classical Shift Logic:** The code implements the mathematical formula $C_i = (P_i + K_i) \pmod{26}$ for encryption, while for decryption it uses $P_i = (C_i - K_i + 26) \pmod{26}$. This ensures that all results stay within the 26-letter alphabet.
- Case Sensitivity Handling:** The loops check for uppercase letters using `isupper(c)` to determine the base character ('A' or 'a'). This method keeps the original casing of the plaintext intact during the calculations.
- Key Rotation:** By using the expression `keyIndex % keyLen`, the keyword can repeat as needed. For example, with the key **KEY** and the message **HELLO**, each letter is shifted accordingly: 'H' is shifted by 'K', 'E' by 'E', and 'L' by 'Y', with the process restarting so that the second 'L' is shifted once more by 'K'.
- Alphabetical Validation:** The code employs the `isalpha(c)` check to ensure that only letters are processed, effectively excluding numbers, spaces, and symbols from the encryption process.
- Non-Alphabetic Preservation:** An `else` block handles blank spaces and special characters with the statement `ciphertext += c`; so that they are added to the result unchanged.
- Smart Key Indexing:** Placing `keyIndex++` within the `isalpha` check means the key only advances when a letter is successfully encrypted. This ensures that the key aligns perfectly with the text, even when non-letter characters are present.
- Dynamic Input Reading:** The program uses `getline(cin, plaintext)` instead of `cin >> plaintext`, allowing it to read entire sentences, including spaces, rather than stopping at the first word.

8. **Buffer Management:** Before calling **getline**, the use of **cin.ignore()** helps clear the "newline" character left in the input buffer from previous menu selections, ensuring the program doesn't skip over user input prompts.
9. **Decryption Correction:** In the decryption function, the code adds + 26 before applying the modulo operator. This helps avoid a "**negative modulo**" result in C++ when the key's value exceeds that of the ciphertext letter.
10. **Modular Structure:** The program is divided into three clear sections—encryption, decryption, and a main menu loop—making it easier to navigate, reuse, and debug the codebase.

2. Output:

```
===== Vigenere Cipher Program =====
1. Encrypt
2. Decrypt
3. Exit
Enter your choice (1-3): 1

--- Encryption ---
Enter plaintext: attackatdawn
Enter key: lemon

Result:
Plaintext: attackatdawn
Key: lemon
Ciphertext: lxfopvefrnhr

===== Vigenere Cipher Program =====
1. Encrypt
2. Decrypt
3. Exit
Enter your choice (1-3): 2

--- Decryption ---
Enter ciphertext: lxfopvefrnhr
Enter key: lemon

Result:
Ciphertext: lxfopvefrnhr
Key: lemon
Plaintext: attackatdawn

===== Vigenere Cipher Program =====
1. Encrypt
2. Decrypt
3. Exit
Enter your choice (1-3): 3

Thank you for using Vigenere Cipher!
```

3. Classical Vignere Cipher Table:

Plain →	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Key ↓																										
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

4. Observation:

The code uses the Vigenère cipher to shift each letter based on the key provided. It wraps around using modulo 26. The code keeps the original case of letters (uppercase and lowercase) and ignores non-letter characters like spaces, punctuation, or numbers. The key repeats as needed to match the length of the text, and only letters in the text change the key index—non-letters do not affect it. Users can choose to encrypt or decrypt text using a key they supply through a simple menu. The decryption function works by subtracting the key's shift, allowing users to recover the original message.

5. Results:

When users choose to encrypt, the program takes a plain text string and a key, then produces the encrypted text. For example, "Hello World" with the key "KEY" turns into "Rijvs Uyvjn." To decrypt, the program uses the same key to retrieve the original text from the encrypted message, showing that the cipher works both

ways. The code handles mixed-case input well and keeps the case in the output. If the key includes non-letter characters, their position in the alphabet can cause a shift, leading to unexpected results. However, the program works well as long as the key mainly includes letters. It continues running until the user exits, providing a straightforward command-line interface for using the cipher.

6. Advantages:

- **Simple Implementation:** The Vigenère cipher is easy to grasp and implement with basic math and character handling.
- **Polyalphabetic Substitution:** Unlike the Caesar cipher, it uses various shifts based on the key, making it tougher to analyze simply.
- **Preservation of Non-Letters:** The code keeps spaces, punctuation, and digits intact, so the message's structure remains the same.
- **Case Preservation:** Uppercase and lowercase letters are handled differently, so the text's original formatting is upheld.

7. Disadvantages:

- **Vulnerability to Known-Plaintext Attacks:** If someone knows part of the plaintext, they might be able to uncover the key or the rest of the message.
- **Key Length Limitation:** The security of the cipher depends on not reusing the key for long messages; short keys can create patterns that undermine the cipher's security (like Kasiski examination).
- **Lack of Integrity/Authentication:** The cipher ensures confidentiality but does not protect against tampering.
- **Alphabetic Key Requirement:** The implementation expects the key to include only letters. Non-letter characters in the key create shifts that can be confusing and might lead to errors.

8. Conclusion:

This C++ program shows how the Vigenère cipher works for encrypting and decrypting messages. It allows users to easily input text while keeping the case and non-letter characters unchanged, and it uses the key correctly. Although the Vigenère cipher is historically important and simple to use, it is not secure enough for today's needs because it is vulnerable to pattern analysis and known-plaintext attacks. Still, this program is a useful tool for learning about basic ciphers and the importance of managing keys in cryptography.

THE END
